

Fundamentos de Procesadores Superescalares

Arquitectura de computadores - Semana 7

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Objetivos

General

Conocer los fundamentos de los procesadores superescalares, analizando su arquitectura, modelos de planificación estática y dinámica

Específicos

- Analizar las **limitaciones del pipeline clásico** (dependencias y riesgos) como motivación para las arquitecturas superescalares.
- Explicar el principio de **emisión múltiple y su impacto en el rendimiento**.
- Comparar **modelos de procesadores superescalares**: cauces independientes vs. etapas con emisión múltiple.
- Diferenciar **planificación estática y dinámica** y el papel del compilador frente al hardware.
- Reconocer las **limitaciones del paralelismo** a nivel de instrucción y sus implicaciones en el diseño del procesador.

Motivación de los procesadores superescalares



Arquitecturas estudiadas hasta ahora

Evolución en las arquitecturas

- Los **procesadores secuenciales** procesan únicamente una instrucción a la vez, lo que lleva a un aumento en el número de ciclos requeridos para ejecutar todo el programa.
- Por otro lado, los **procesadores segmentados** definen un único cauce con varias etapas (IF/ID/EX/MEM/WB) que permiten iniciar el procesamiento de una instrucción antes de que la instrucción actual haya sido completamente procesada.
- La eficiencia de este **procesamiento no es óptimo** debido a las dependencias entre instrucciones que pueden generar cuellos de botella.

¿Cómo aumentar aún más el rendimiento del procesador?

1. Aumentar la velocidad del reloj

- Limitado por la tecnología y la disipación de energía

2. Aumentar la profundidad del pipeline

- Superponer el procesamiento de instrucciones a través de un pipeline más profundo
 - Por ejemplo, pasar de 5 etapas a 10 o 15 etapas
 - Menos trabajo por cada etapa
 - Implicaría un ciclo de reloj más corto y menor consumo de energía
 - Sin embargo, más probabilidad de que se produzcan los tres tipos de riesgos y haya más bloqueos, implicando un $CPI > 1$

3. Diseñar un procesador superescalar

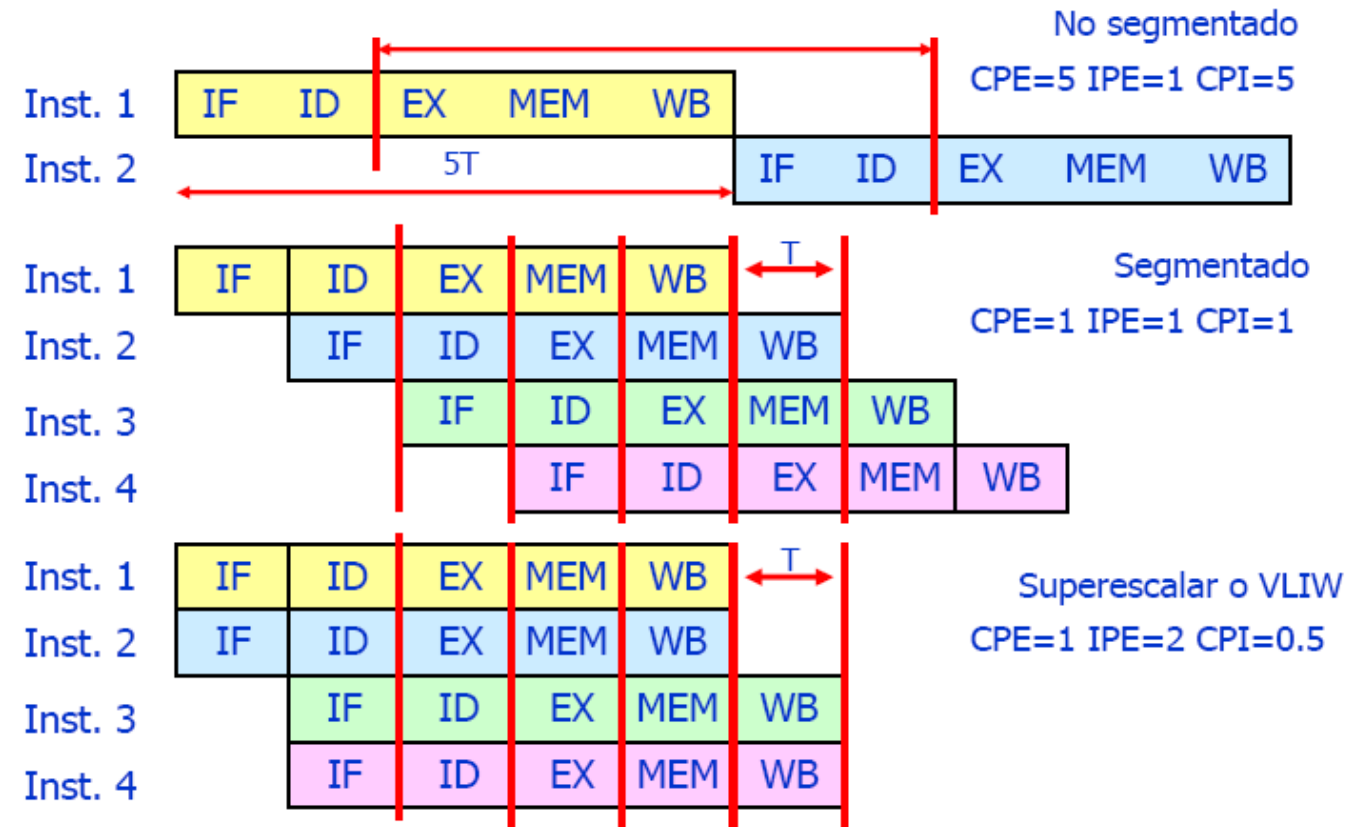
- Un procesador con una implementación capaz de procesar más de una instrucción, en la misma etapa, por ciclo de reloj.

Ciclos Por Instrucción

- La emisión de instrucciones se refiere al proceso en el cual una unidad de procesamiento (como una CPU) toma instrucciones de un flujo de instrucciones y las coloca en su etapa inicial de ejecución en el pipeline.
 - Ciclos Por Emisión (CPE)
 - Instrucciones Por Emisión (IPE)

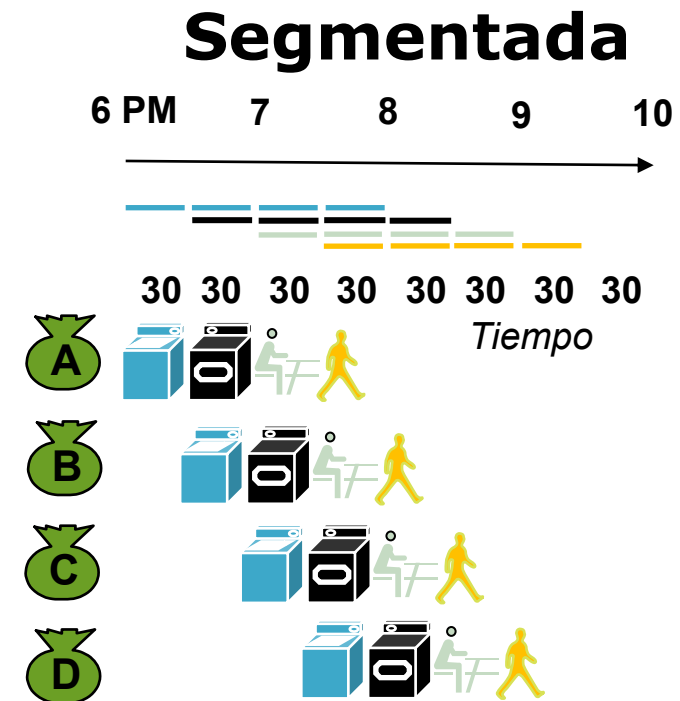
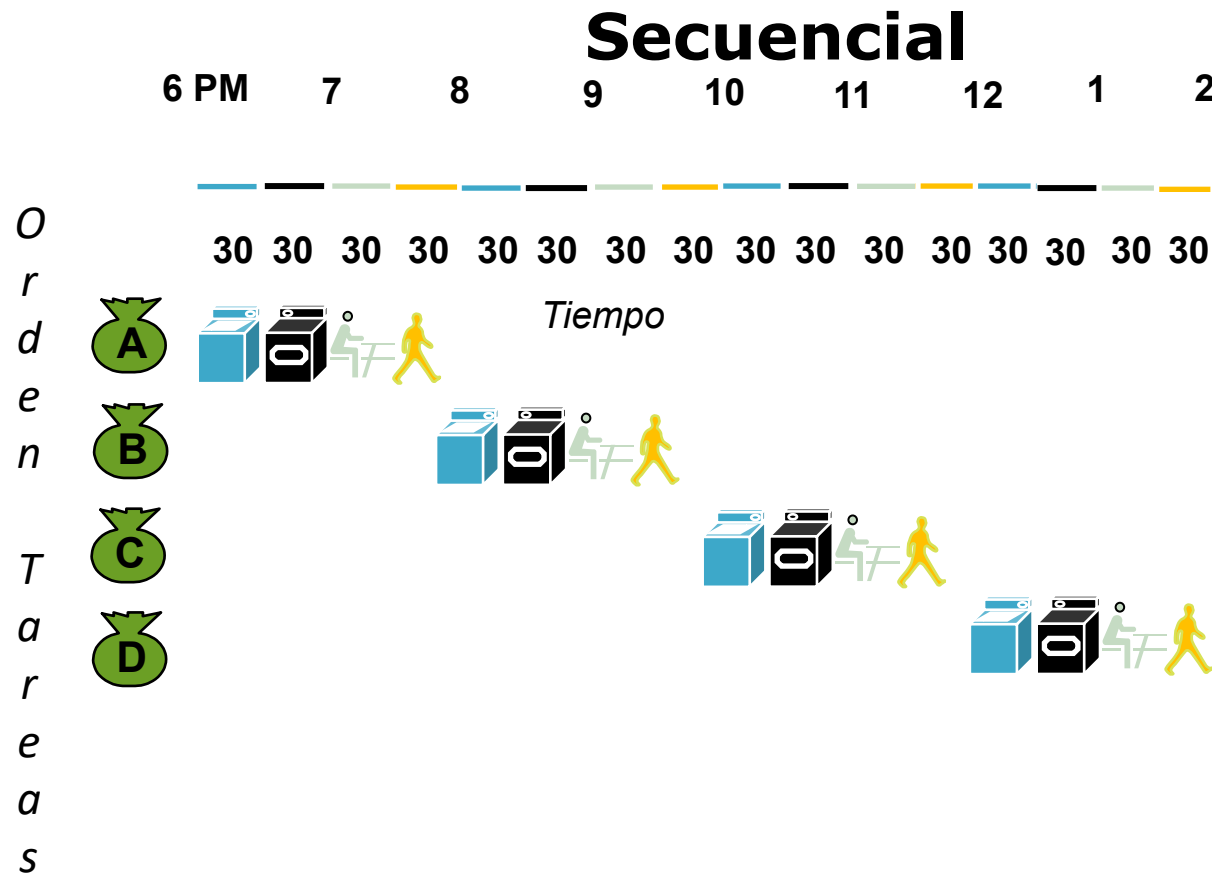
$$CPI = \frac{CPE}{IPE}$$

$$T_{CPU} = NI \cdot \frac{CPE}{IPE} \cdot T_{Ciclo}$$

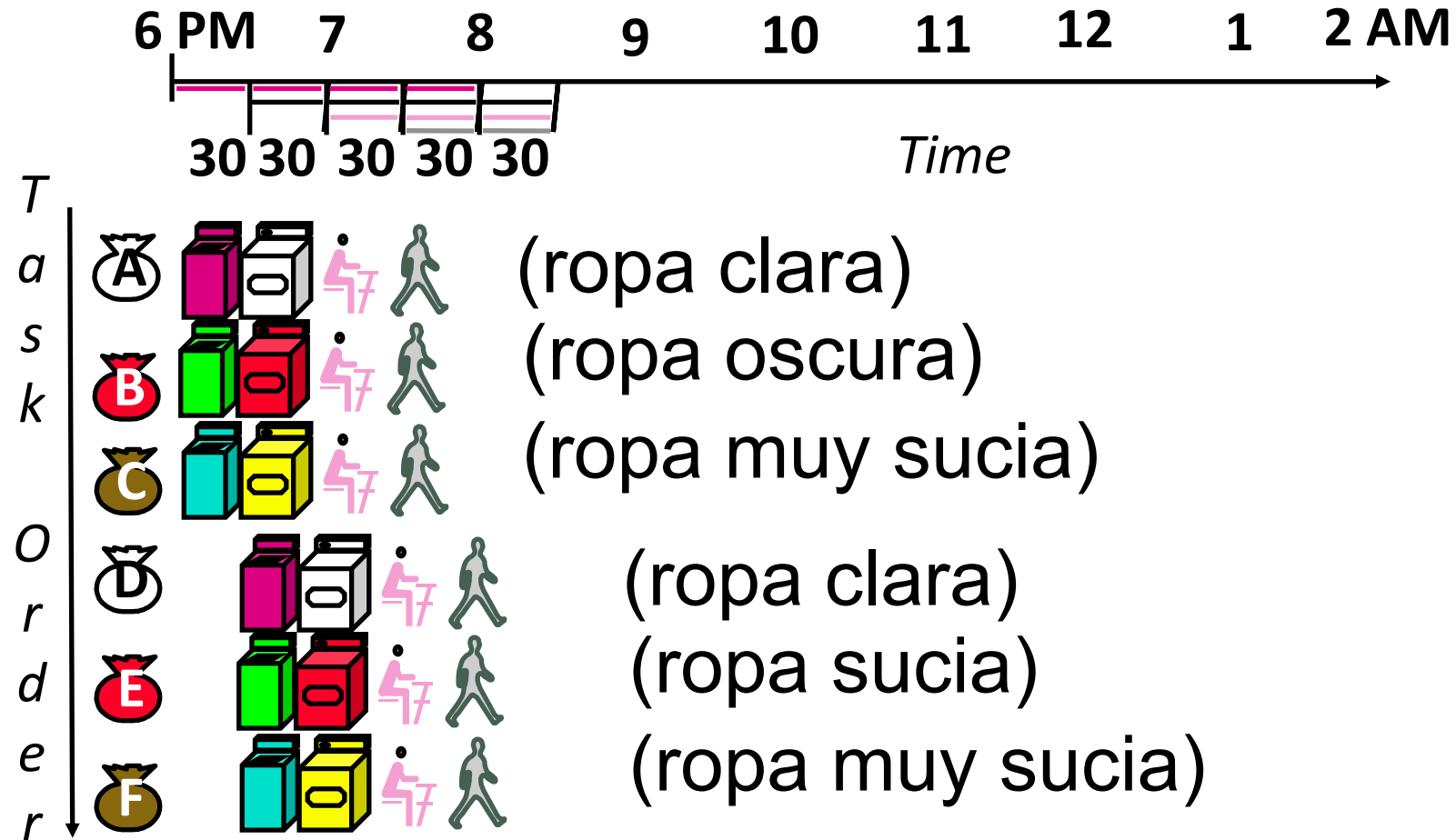


Ejemplo ilustrativo: Domingo de colada

- Cada cesta de ropa implica 4 tareas:
 - 1) Lavar, 2) Secar, 3) Doblar y 4) Colocar en los cajones



Ejemplo ilustrativo: Domingo de colada



Se necesitan más recursos para realizar tareas paralelas

Procesadores superescalares

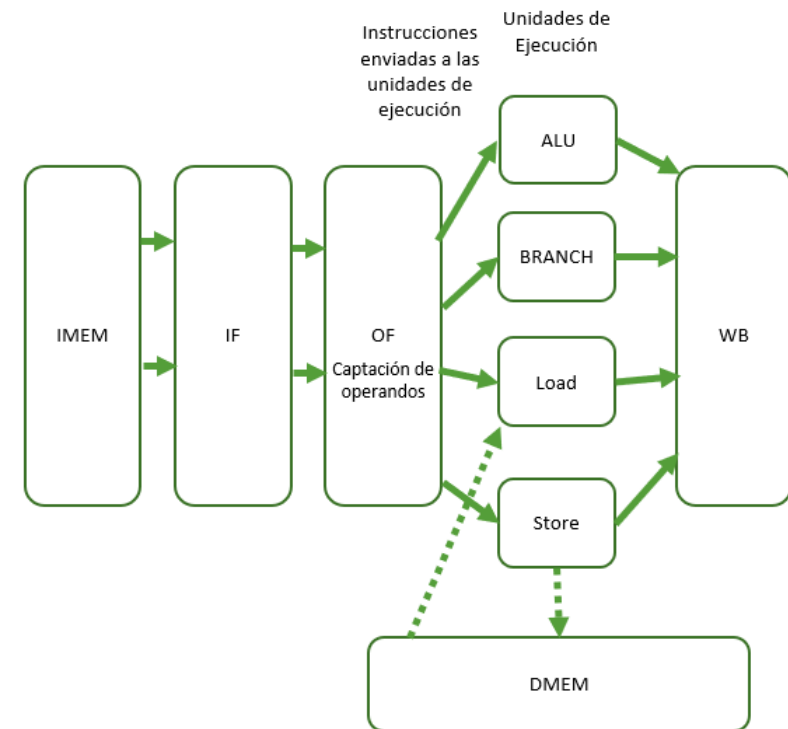
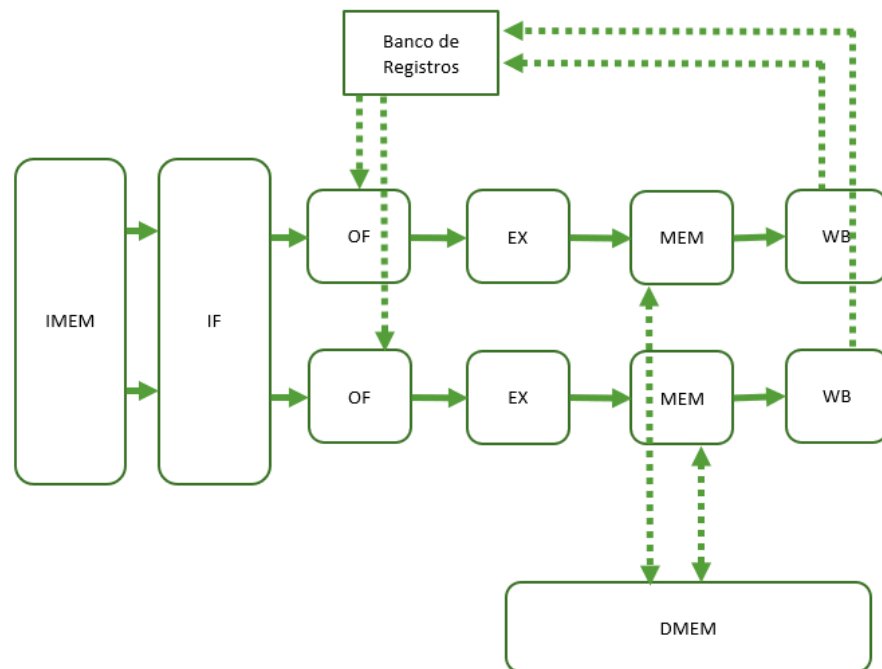
- La arquitectura superescalar se refiere a una implementación capaz de procesar **múltiples instrucciones** en un solo ciclo de reloj, lo cual se logra mediante el uso de **múltiples cauces**.
 - Se permite que varias instrucciones comiencen su ejecución de manera **independiente**.
 - Se emplean **múltiples unidades funcionales** que pueden **ejecutar instrucciones en paralelo** y de manera **independiente**.
- Consideraciones importantes de estos procesadores:
 - Los procesadores superescalares tienden a presentar una **microarquitectura más compleja**.
 - Su diseño a menudo requiere la incorporación de **estructuras adicionales** que permiten el almacenamiento temporal de la información necesaria para el procesamiento de múltiples instrucciones en paralelo.
 - Es esencial considerar la **sincronización entre las diversas etapas** del procesador, lo que puede resultar en una mayor complejidad de diseño.

Modelos de procesadores superescalares



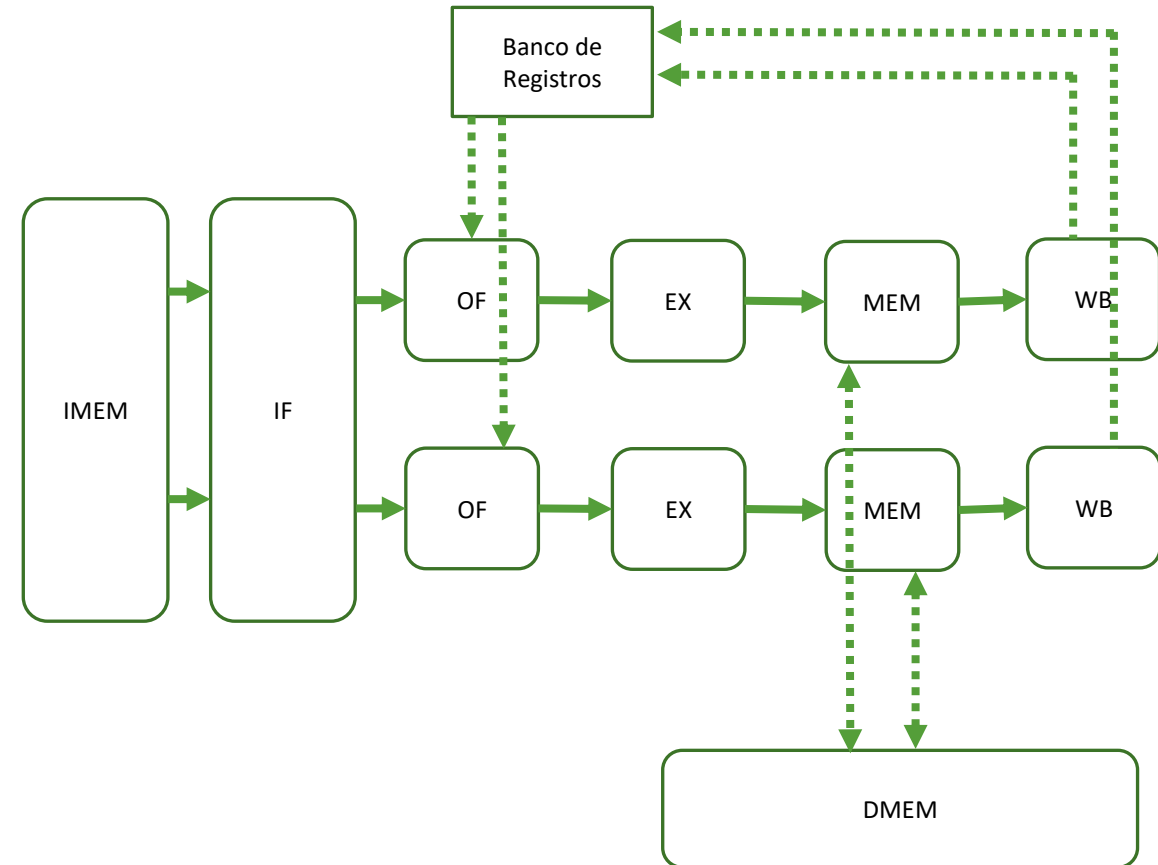
Modelos de procesadores superescalares

- Los procesadores superescalares tienen la capacidad de emitir, decodificar, ejecutar y completar varias instrucciones en cada ciclo de reloj, lo que los hace ideales para aumentar el rendimiento.
- Dos modelos de procesadores superescalares diferentes:
 - 1) Cauces totalmente independiente (izquierda)
 - 2) Cauce donde cada etapa procesa varias instrucciones (derecha)



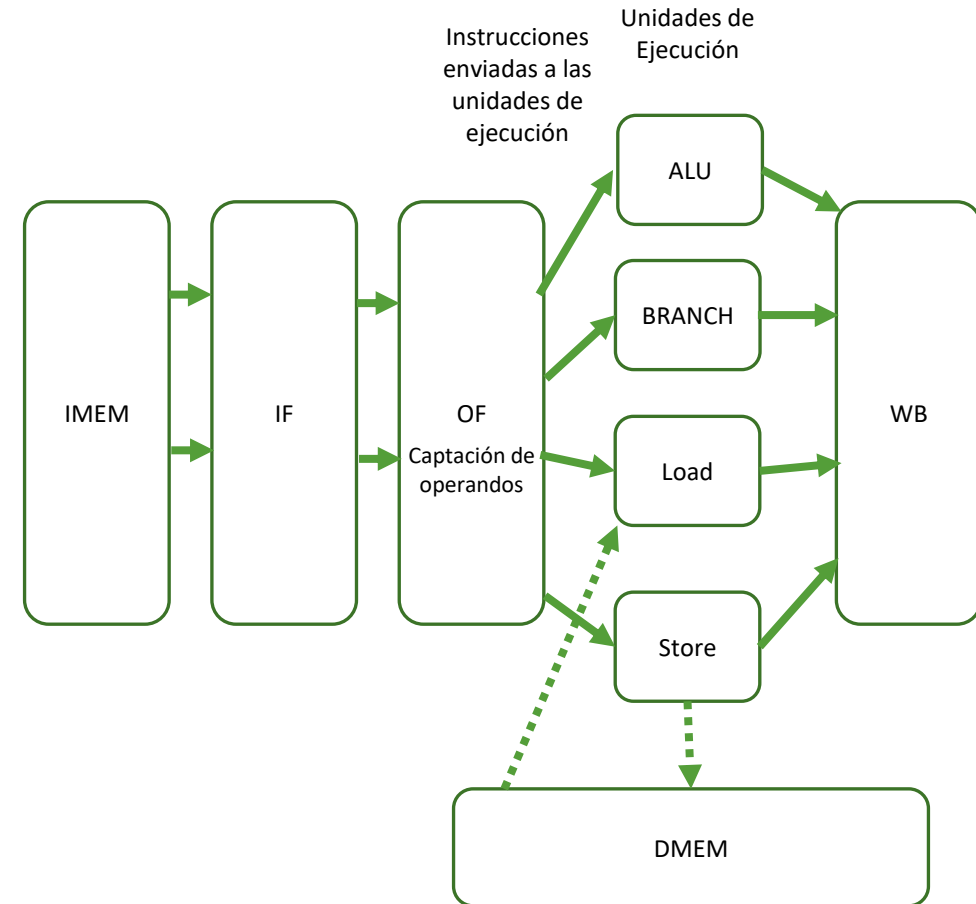
Modelos de procesadores superescalares

- **Dos cauces totalmente independientes:**
 - En este modelo, el procesador tiene **dos cauces** (pipelines) completamente independientes.
 - Cada uno de estos cauces está diseñado para **manejar un tipo específico de instrucción**.
 - Por ejemplo, un cauce puede estar especializado en instrucciones de punto flotante, mientras que el otro se enfoca en instrucciones enteras.
 - Esta especialización permite que el procesador maneje diferentes tipos de instrucciones de **manera eficiente**.
 - Sin embargo, para aprovechar al máximo esta configuración, **el código debe estar muy bien organizado en pares de instrucciones** que puedan ser ejecutadas en paralelo por los dos cauces.



Modelos de procesadores superescalares

- **Un cauce donde cada etapa procesa varias instrucciones:**
 - El procesador tiene un solo cauce, pero cada **etapa** de este cauce es capaz de procesar **varias instrucciones simultáneamente**.
 - Para lograr esto, se requieren **estructuras de almacenamiento** entre las etapas para gestionar múltiples instrucciones en diferentes estados de ejecución.
 - Ej. En una etapa de decodificación, se pueden decodificar varias instrucciones en paralelo, estas instrucciones avanzan a través de las etapas siguientes.
 - **Aumenta la eficiencia** de la ejecución y mejora el **rendimiento**.
 - Este es el modelo más común en los procesadores superescalares.



Procesador superescalar

- **El orden de captación y decodificación de la instrucciones en el flujo del programa es inalterable.**
 - Sin embargo la emisión para la ejecución y la escritura en memoria/registros puede ser ordenada o desordenada.
- **Desordenando la emisión de las instrucciones** se intenta obtener el mayor grado de paralelismo de instrucción y de procesador.
- La única restricción es que el resultado del programa sea correcto.
- Se requiere un procesador muy sofisticado.

Planificación de la emisión de múltiples instrucciones



Planificación de la emisión de múltiples instrucciones

- **Emisión múltiple estática → protagonista el software**
 - Planificación estática
 - El compilador agrupa las instrucciones que deben emitirse juntas antes de la ejecución del programa.
 - Las empaqueta en “paquetes de emisión” – “issue slots”
 - El compilador detecta y evita peligros/riesgos
- **Emisión múltiple dinámica → protagonista el hardware**
 - Planificación dinámica
 - La CPU examina el flujo de instrucciones y elige las instrucciones que emitirá en cada ciclo
 - El compilador puede ayudar reordenando las instrucciones
 - La CPU resuelve los peligros mediante técnicas avanzadas en tiempo de ejecución

Planificación estática

- Una CPU superescalar con planificación estática se caracteriza por la mejora o replicación de sus unidades funcionales, lo que le permite ejecutar múltiples instrucciones en una misma etapa y ciclo de reloj.
- **El orden en el que las instrucciones aparecen en el programa desempeña un papel crucial para aprovechar al máximo esta arquitectura.**
 - La CPU procesará las instrucciones en el mismo orden que en el código del programa.
 - Es fundamental **organizar las instrucciones** de tal manera que se puedan aprovechar al máximo las capacidades de la arquitectura.
- El **compilador** es la herramienta responsable de organizar las instrucciones en el código ensamblador de la manera más eficiente para una CPU específica.
 - Las **opciones de optimización del compilador** analizan las interacciones entre las instrucciones y las unidades funcionales de cada etapa, además de evaluar los posibles riesgos de datos y control.
- El objetivo es generar el **código más eficiente y optimizado** posible para aprovechar al máximo el rendimiento de la CPU.

Planificación estática

- **Ejemplo**

- Se dispone de una única unidad de transferencia a memoria de datos bidireccional (hacia y desde memoria).
 - Es esencial evitar que en la secuencia de instrucciones se encuentren dos operaciones de este tipo (transferencia a memoria) próximas.
 - En caso de ocurrir, la CPU se verá obligada a retrasar una de ellas mediante el procedimiento comúnmente conocido como "detención" o "stall".
-
- El desarrollo de código ensamblador para una CPU superescalar con planificación **estática demanda un profundo conocimiento de la arquitectura** por parte del programador.

Planificación estática

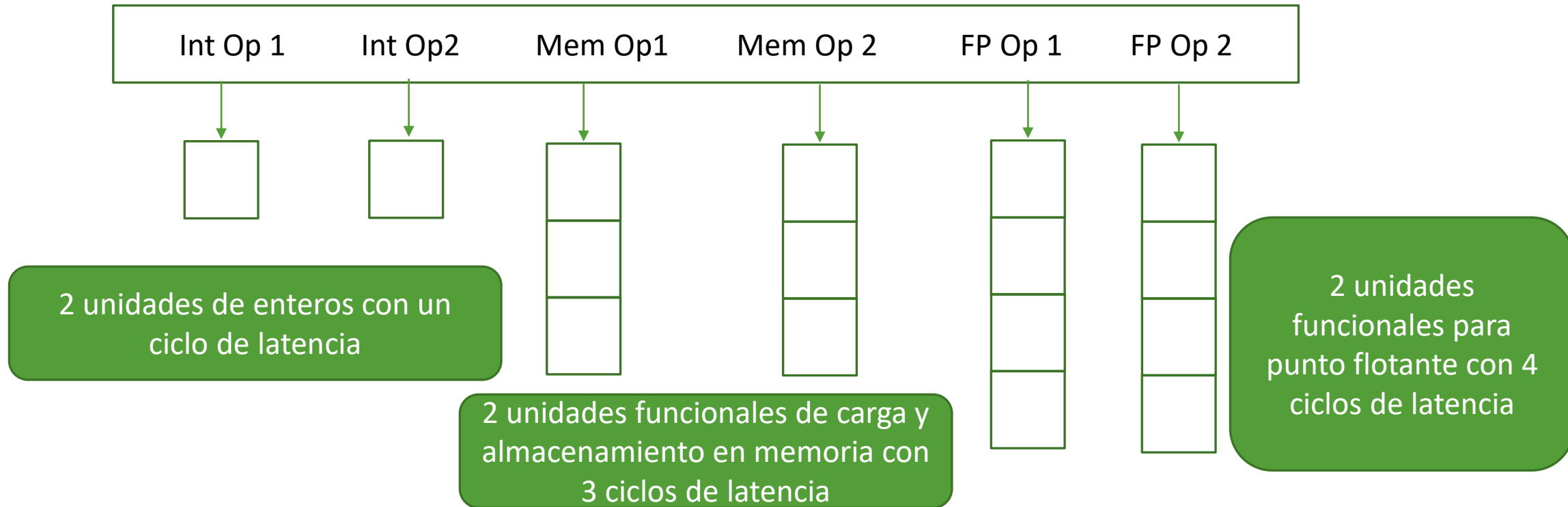
- **Claves**

- El compilador organiza las instrucciones en grupos de emisión.
- Estos grupos contiene instrucciones independientes que pueden ejecutarse en un solo ciclo.
- La capacidad de agrupamiento depende de los recursos del cauce disponibles.
- El compilador debe abordar la eliminación de riesgos y la reordenación de instrucciones en los grupos de emisión.
- Se busca minimizar las dependencias dentro de un grupo, aunque puedan existir algunas entre grupos.
- Este enfoque puede variar según las Arquitecturas de Conjunto de Instrucciones (ISA); el compilador debe tener este conocimiento.
- En caso necesario, el compilador inserta instrucciones "nop" para completar los grupos de emisión.

Planificación estática

- **Arquitecturas relacionadas: VLIW**

- La arquitectura VLIW (Very Long Instruction Word) es un tipo de arquitectura de procesador que se diseñó para permitir la ejecución de múltiples instrucciones en paralelo. Las instrucciones se agrupan en "palabras largas de instrucción" y se espera que el software especifique explícitamente qué instrucciones pueden ejecutarse en paralelo.



Planificación estática

- **Arquitecturas relacionadas: VLIW**
- **Ventaja:** alto grado de control al programador o al compilador, lo que resulta una ejecución altamente eficiente de instrucciones.
- **Limitaciones importantes:**
 - **Dependencia del compilador para detectar y programar instrucciones** que pueden ejecutarse en paralelo. Si el compilador no puede generar código optimizado, la arquitectura VLIW no proporcionará un rendimiento óptimo.
 - **Dificultad de programación y propenso a errores**, ya que el programador o el compilador deben conocer y gestionar las dependencias entre instrucciones de manera explícita.
 - **Falta de flexibilidad**, ya que las instrucciones se agrupan en palabras largas de instrucción y se programan estáticamente, la arquitectura VLIW puede carecer de flexibilidad para adaptarse a cargas de trabajo cambiantes.

Planificación dinámica

- Una arquitectura superescalar con planificación dinámica presenta un diseño de procesador más sofisticado en comparación con la planificación estática. Para lograr su funcionamiento de manera óptima, requiere de componentes esenciales que incluyen:
 - **Cola de instrucciones.** Esta estructura de datos almacena las instrucciones transferidas desde la memoria, es decir, instrucciones que han sido captadas.
 - **Ventana de instrucciones.** Constituida por una o más estructuras de datos (también denominadas estación de reserva) que mantienen instrucciones en espera, listas para recibir sus operandos y la asignación de una unidad funcional adecuada para su ejecución. Contiene instrucciones que han sido decodificadas y están esperando para ser emitidas y ejecutadas.
 - **Buffer de reordenamiento (ROB).** Compuesto por una o más estructuras de datos. Este componente garantiza que la escritura de resultados, ya sea en registros o en memoria, se realice de manera coherente y secuencial (consistencia)
 - **Predictor de saltos (BTB).** Este elemento desempeña un papel crítico al reducir el impacto de los riesgos de control ya que, en una arquitectura superescalar, los costes asociados a estos riesgos son significativamente mayores en comparación con una arquitectura no superescalar.

Planificación dinámica

- Gracias a estas estructuras de datos y al hardware adicional, diseñado para tomar decisiones basadas en la información que albergan, la CPU superescalar tiene la capacidad de **enviar a ejecutar (emitir) instrucciones de forma desordenada (fuera de orden)**.
- En términos generales, la política de emisión fuera de orden se fundamenta en las siguientes condición:
 - *Tan pronto como una instrucción tenga sus operandos disponibles y la unidad funcional necesaria está disponible, se procede a la emisión de la instrucción, siempre que se puedan emitir tantas instrucciones como estén disponibles.*
- Sin embargo, el resultado generado no se transfiere inmediatamente al banco de registros o a la memoria, ya que se espera completar esta fase de manera que se mantenga la **consistencia**
 - *Entraremos más adelante sobre el concepto de consistencia*

Planificación dinámica

- **Claves:**
 - Los procesadores superescalares son capaces de emitir una cantidad variable de instrucciones en cada ciclo y esta decisión recae en la CPU misma.
 - Esta capacidad permite evitar riesgos estructurales y de datos, mejorando la eficiencia de ejecución.
 - Aunque no siempre se elimina por completo la necesidad de intervención del compilador, la CPU asume un papel fundamental en garantizar la semántica correcta del código, facilitando así el proceso de ejecución de instrucciones.

¿Por qué hacer planificación dinámica?

- Permite una ejecución más eficiente de instrucciones y una adaptación a las condiciones cambiantes del flujo de trabajo.
- **Mejor explotación del paralelismo de instrucciones.** La planificación dinámica permite al procesador identificar y aprovechar oportunidades de ejecución en paralelo que pueden no ser evidentes en el código fuente. En contraste, la planificación estática requiere que el programador o el compilador especifique explícitamente las instrucciones paralelas, lo que puede ser limitante y menos flexible.
- **Adaptación a condiciones en tiempo real.** La planificación dinámica puede tomar decisiones en tiempo real sobre qué instrucciones ejecutar, basándose en factores como la disponibilidad de datos, la carga de trabajo actual y las dependencias de las instrucciones. Esto permite una adaptación dinámica y eficiente a las condiciones cambiantes.
- **Optimización de caché.** La planificación dinámica puede optimizar la ejecución de instrucciones para minimizar las penalizaciones de caché. Puede reorganizar las instrucciones para maximizar la reutilización de datos en la memoria caché, lo que reduce la latencia de acceso a la memoria y mejora el rendimiento general.
- **Ejecución de instrucciones especulativas.** En un procesador superescalar con planificación dinámica, es posible ejecutar instrucciones de manera especulativa, es decir, anticiparse a la resolución de ciertas dependencias de datos. Si la suposición resulta ser correcta, esto puede llevar a una ejecución más rápida de instrucciones y un mejor rendimiento.

Recursos

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- Arquitectura de Computadores de Julio Ortega, Alberto Prieto y Mancia Anguita. Ediciones Paraninfo – 978849732274
- Great Ideas in Computer Architecture (Machine Structures) CS 61C at UC Berkeley